IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are

PART I

Collection and Preparation
Lec 2-5

PART II

SQL
Lec 6-8

NoSQL
Lec 9-10

Big Data
Lec 11-13

PART III

Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing

A distributed program

```
(sales.groupBy("product_id")
      .sum("amount")
      .show())
```

A distributed program still needs infrastructure, storage and a way to run again

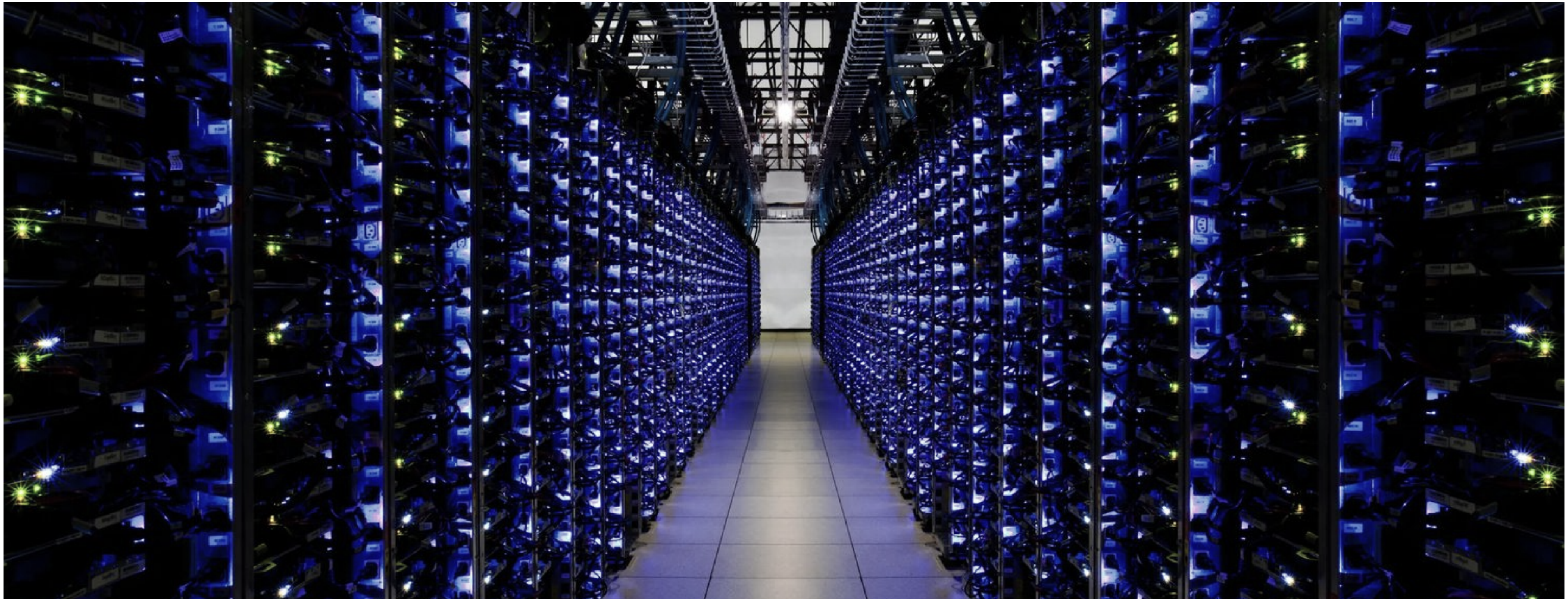
Outline: Big Data II

01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

01 · Cloud and Serverless

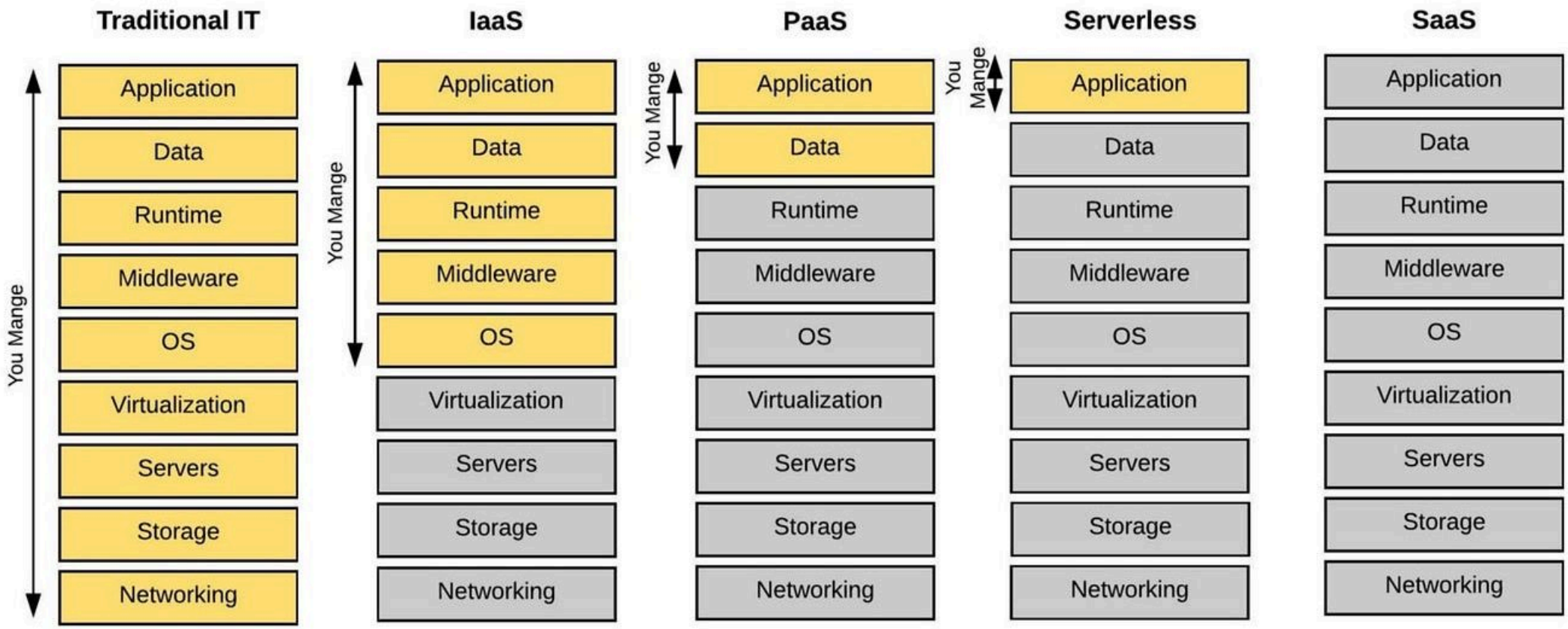
01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Managed Infrastructure



The cloud still runs on physical servers that somebody must operate

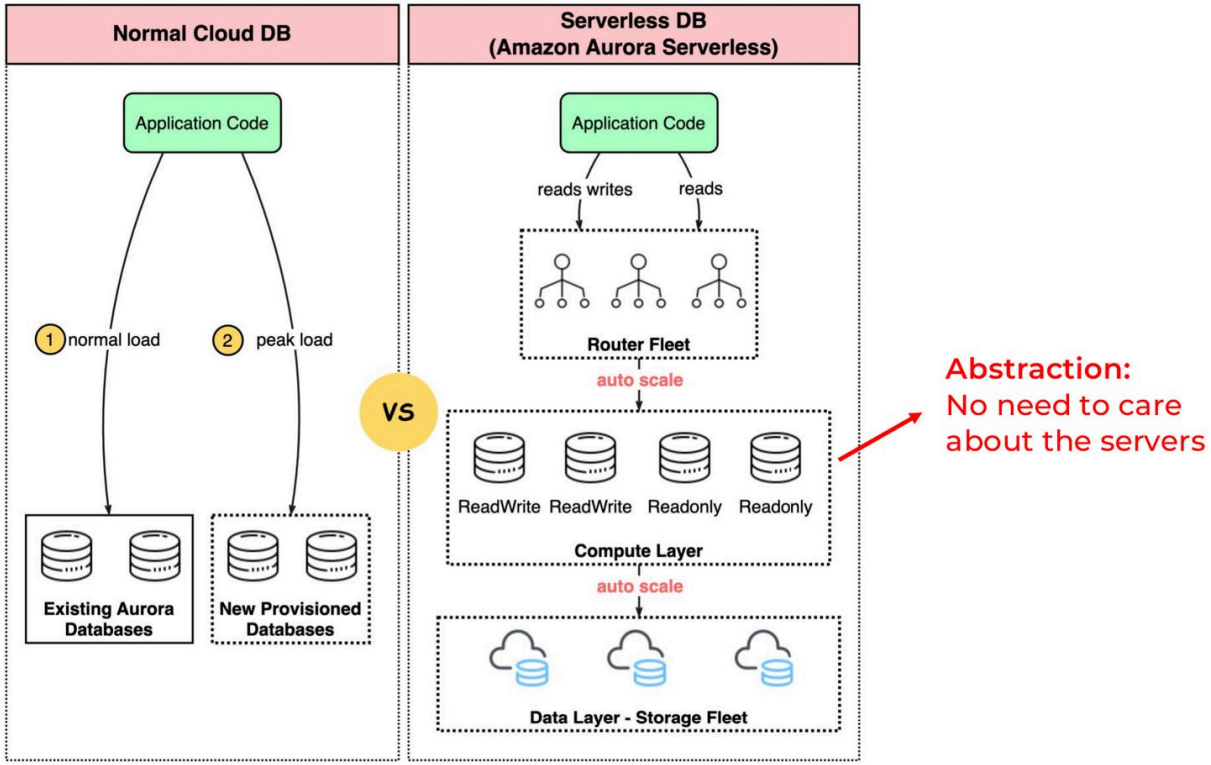
Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service



The service scales servers behind the application interface

Milestones in Managed Analytics

2004
MapReduce paper
batch on clusters

2009
Spark at Berkeley
reusable chains

2010
BigQuery preview
SQL as a service

Today
managed compute
and shared storage

Analytics shifted from operating clusters to submitting SQL to a service

BigQuery as a Managed Service



Ingest

- Batch & Streaming
- Structured & Semi-structured



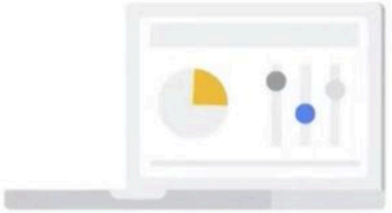
Store

- Up to Petabytes of data
- Datasets & tables
- Federate data



Analyze

- Super-fast SQL queries
- Aggregate
- Transform with DML



Visualize

- Export and connect to other tools more easily

Analysts submit SQL while the service manages distributed execution

Spark and BigQuery

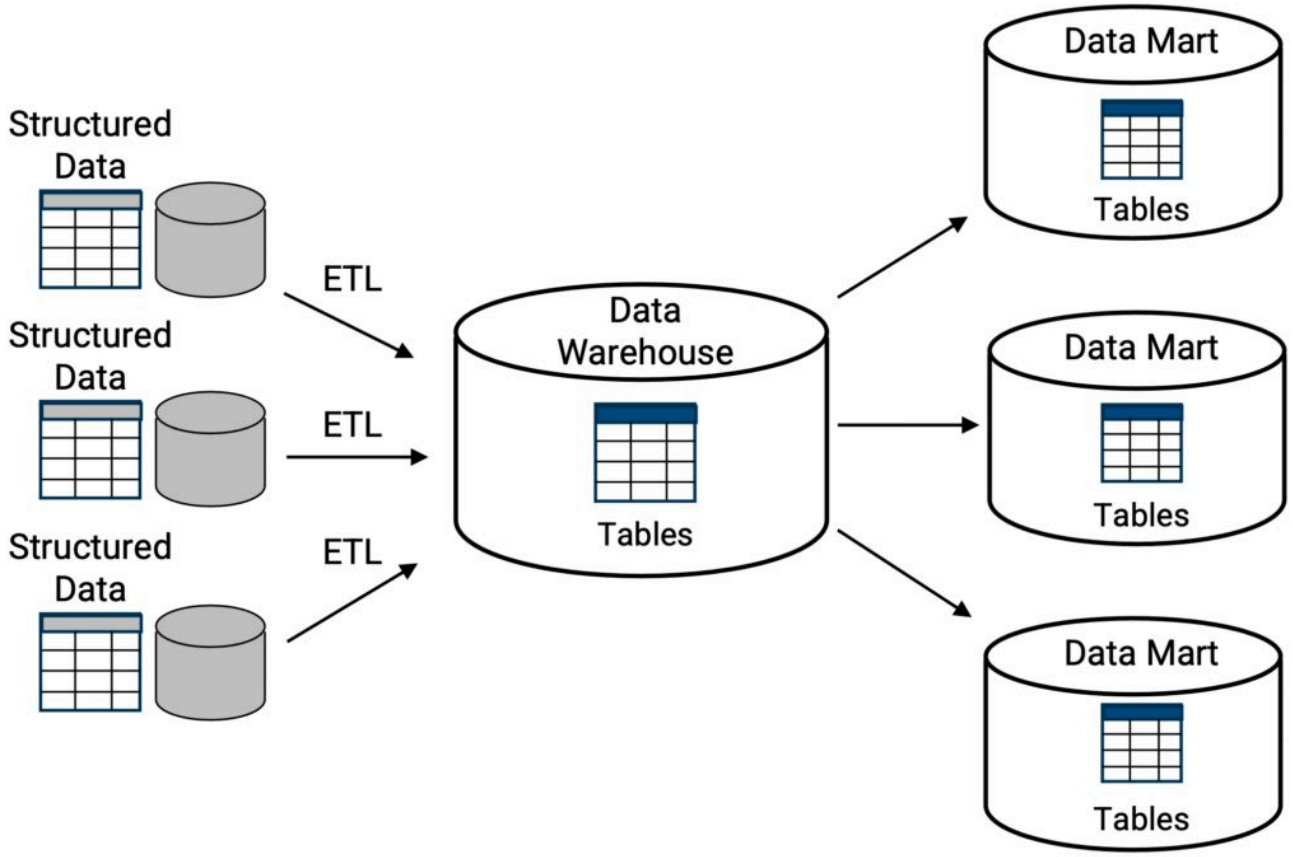
Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

02 · Analytical Warehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

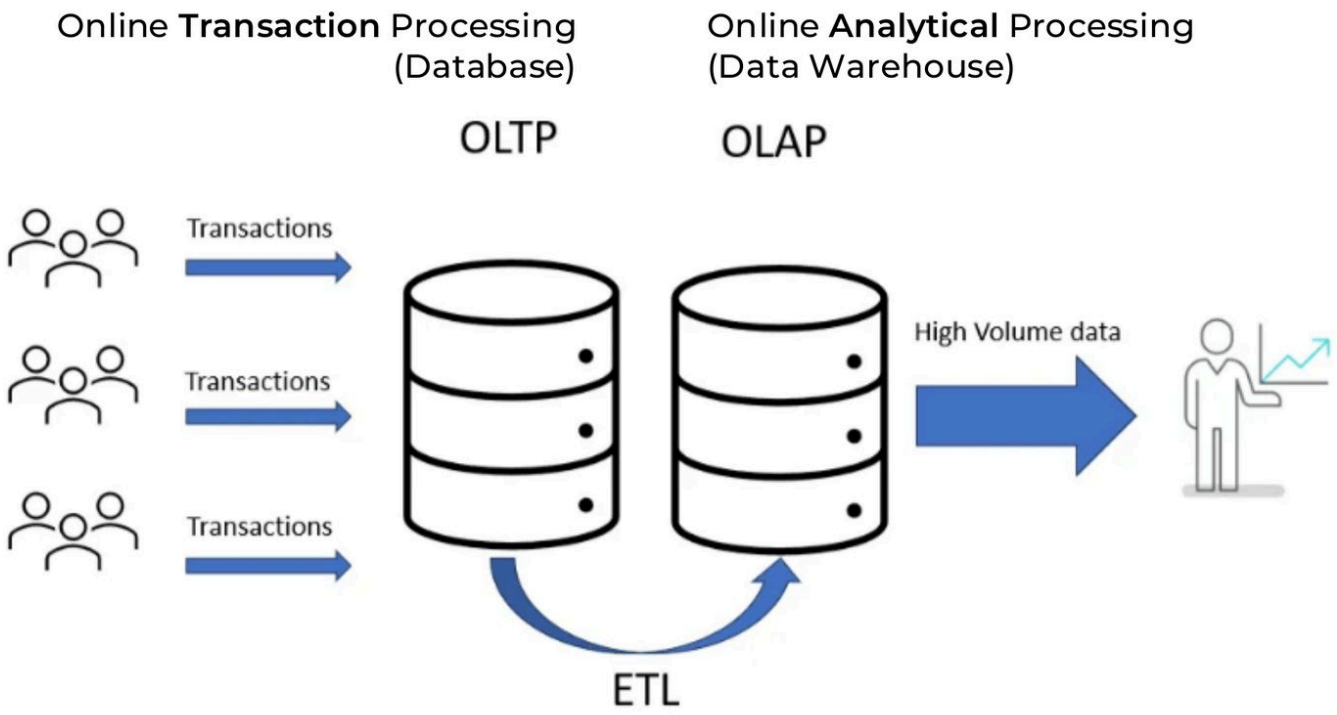
Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



OLTP typically uses row stores; OLAP typically uses column stores

Workloads Shape Physical Storage

Workload	Read / write pattern	Typical physical choice
OLTP	Find and update a few complete records	Row store + key indexes
OLAP	Scan a few columns across many records	Column store + compressed scans
Mixed workload	Fresh writes and large analytical reads	Separate replicas or a hybrid design

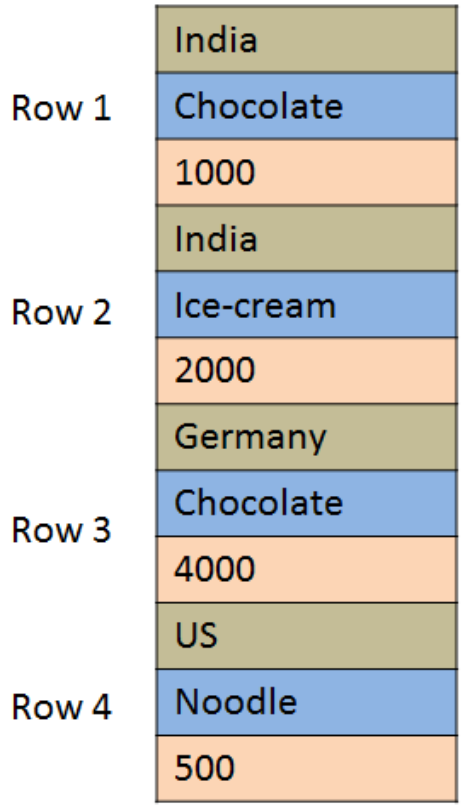
OLTP favors record access; OLAP favors scanning selected attributes

The Same Values on Disk

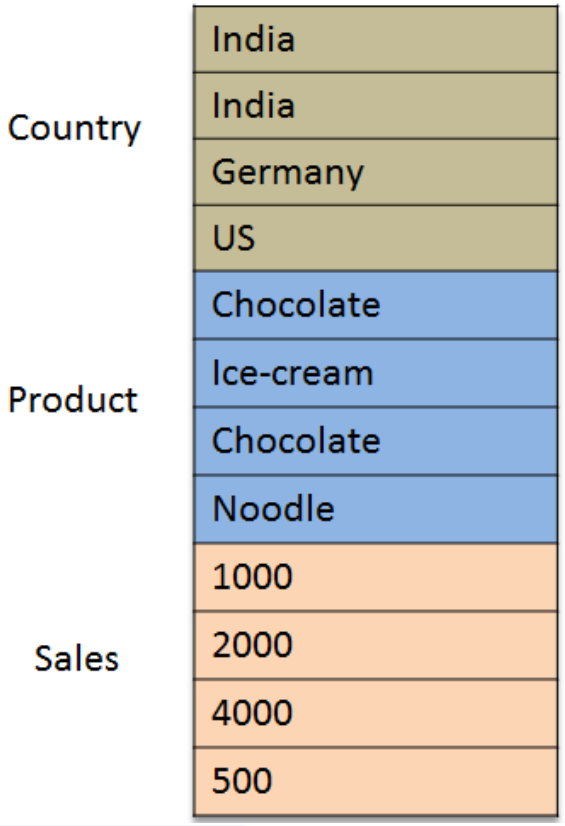
Logical table

Country	Product	Sales
India	Chocolate	1000
India	Ice-cream	2000
Germany	Chocolate	4000
US	Noodle	500

Row store (OLTP)



Column store (OLAP)

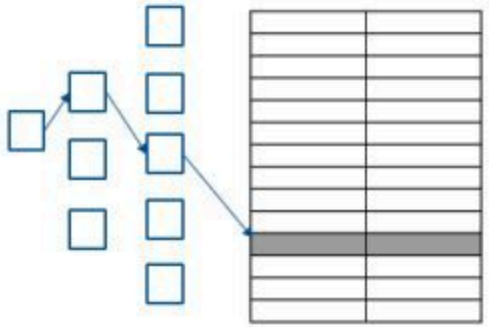


Twelve values retain their meaning while their physical order changes

Columnar Storage

The access pattern explains the usual OLTP / OLAP storage choice

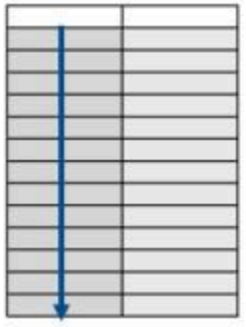
Row-oriented storage & execution



```
SELECT * FROM orders
WHERE orderid = $1
```

At scale: Fast on OLTP, Slow on OLAP

Column-oriented storage & execution



```
SELECT productid, count(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC LIMIT 10
```

At scale: Slow on OLTP, Fast on OLAP

Row stores gather records; column stores scan and compress selected attributes

Read Cost Before Execution

N rows · C columns · K required columns; equal-width values, uncompressed; no index or cache

Partition = a stored group of rows, such as one sale_date

Oct 2	Oct 3	Oct 4
date | product | amount **read**	date | product | amount skip	date | product | amount skip

Date filter selects stored partitions; projection selects columns within them

Scan model	Values read	Assumption
Row	N × C	Read full rows
Column projection	N × K	Include output and filter columns
Partition + projection	f × N × K	f = fraction of rows in scanned partitions

Pruning skips stored row groups; projection skips columns inside the remaining groups

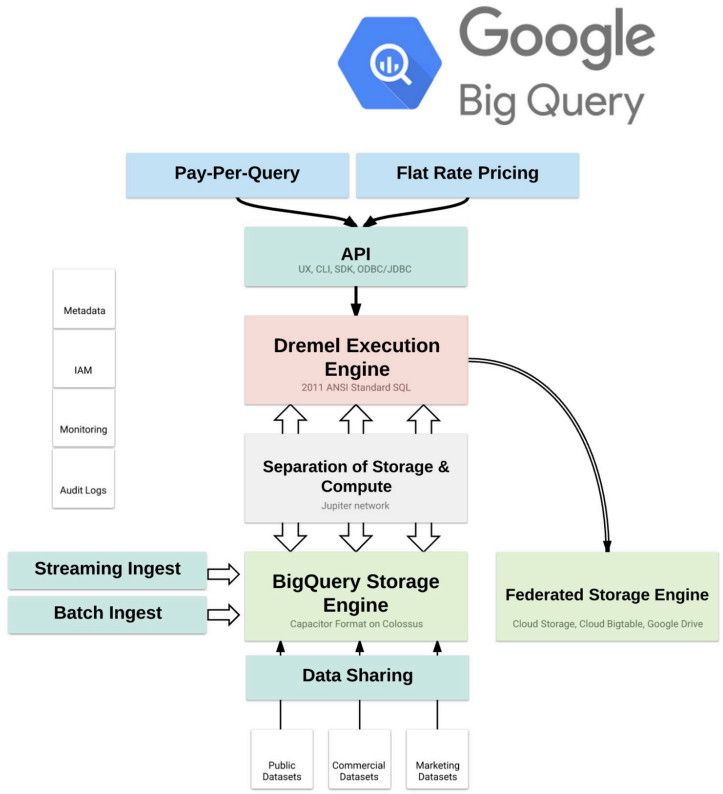
Columns Enable Compact Encoding

Order of product IDs	Run-length encoding	Runs
0, 1, 0, 1, 0, 1	(0,1), (1,1), (0,1), (1,1), (0,1), (1,1)	6
0, 0, 0, 1, 1, 1	(0,3), (1,3)	2

Equal values can compress differently because ordering changes the number of runs

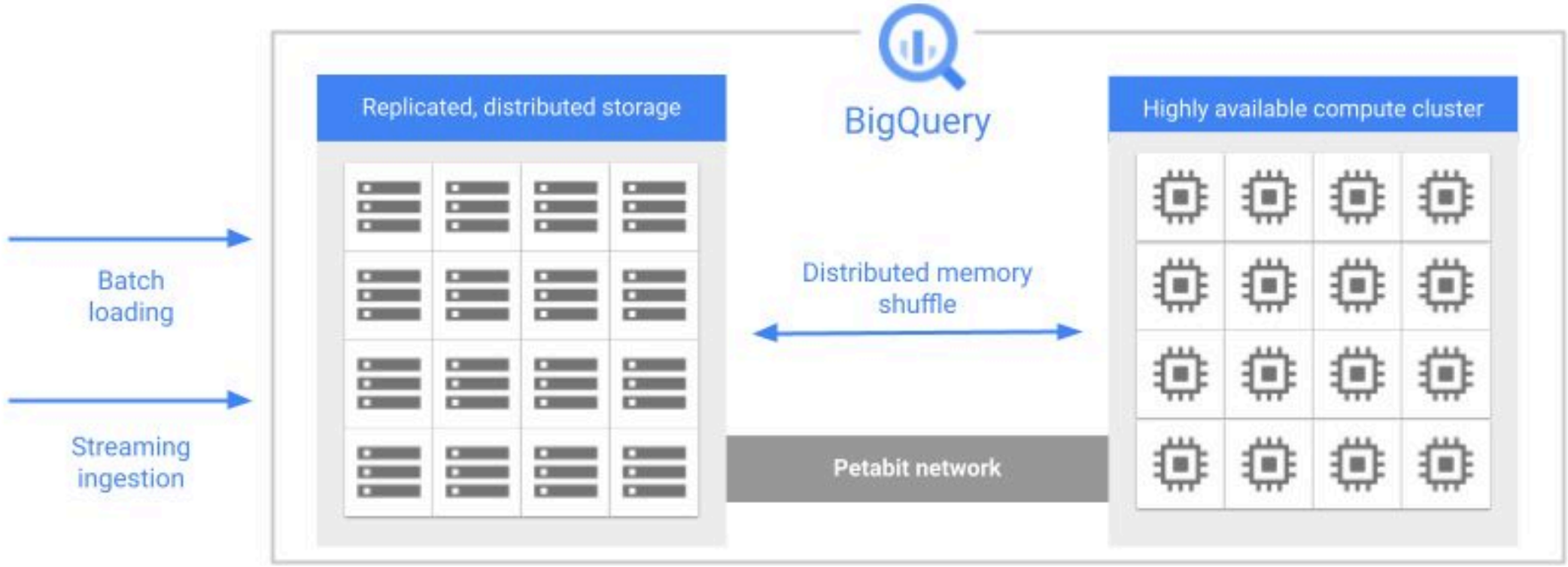
Inside BigQuery

Historical architecture: follow the query and its data



SQL execution, columnar storage and the network form separate service layers

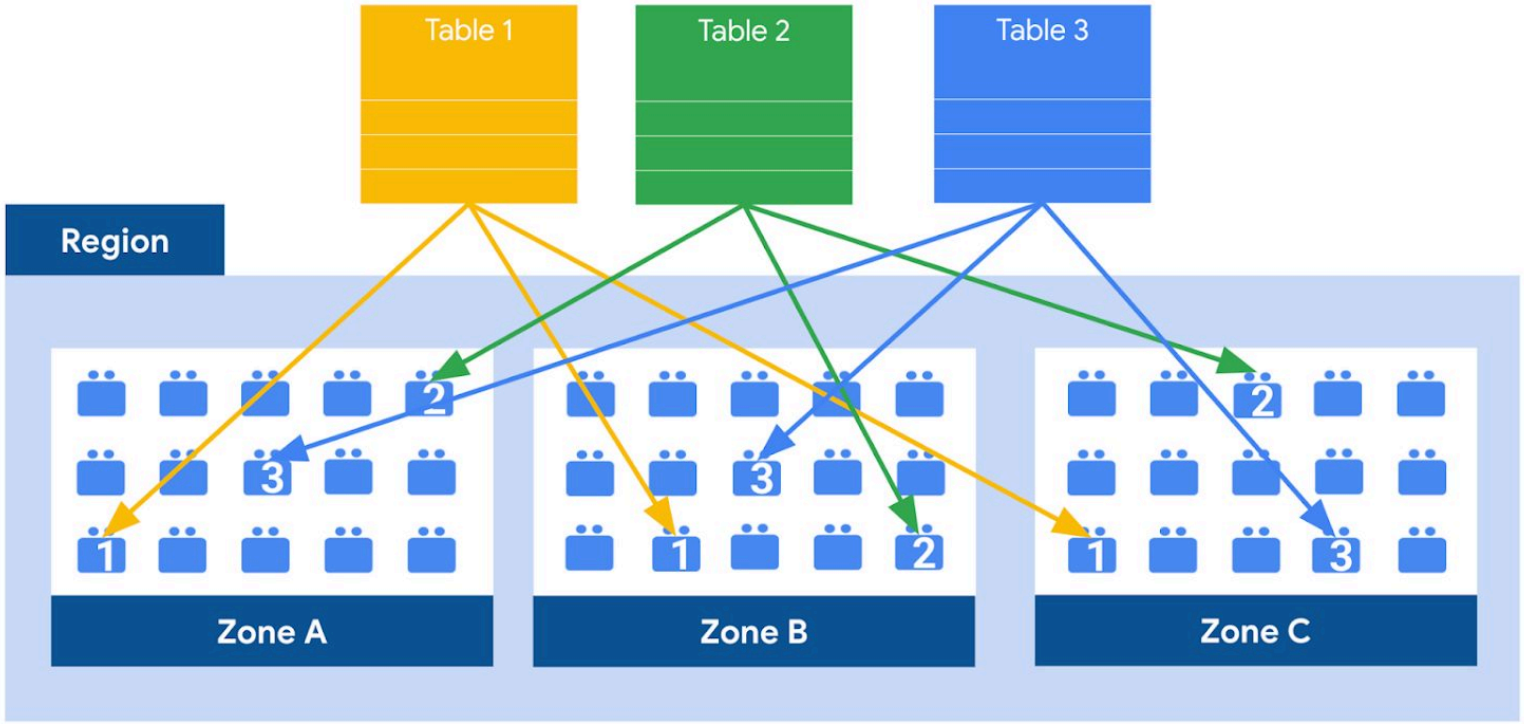
BigQuery Storage and Compute



Persistent table storage and query execution are managed as separate layers

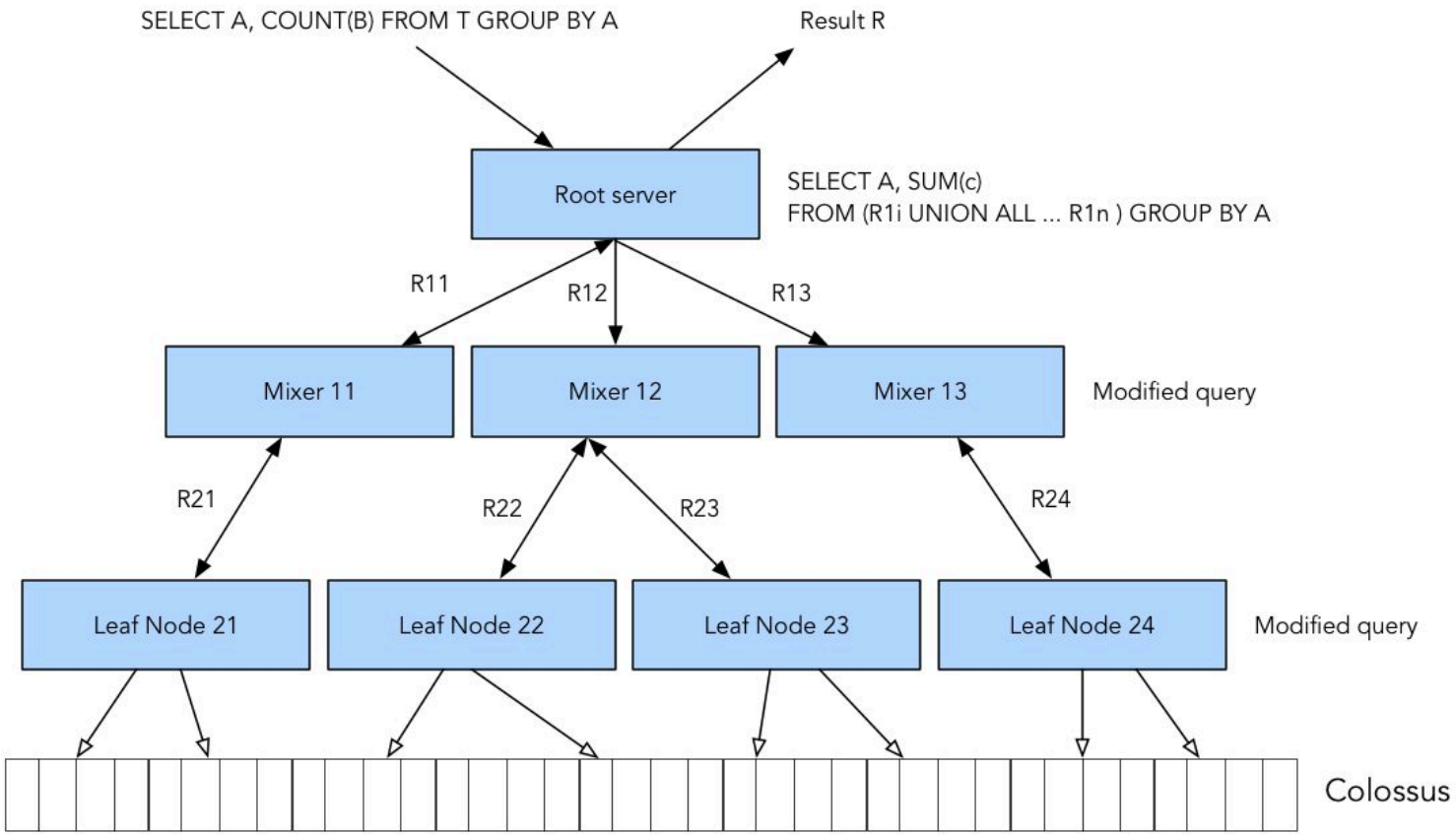
Managed Fault Tolerance

Historical illustration of replicated storage



Replicated storage protects against infrastructure failures rather than incorrect analytical logic

Distributed Query Execution



Workers read portions of the table and combine partial results

Partial Results Must Be Mergeable

Count rows locally, then merge counts by country

Worker	Partial counts	Root merge
A	India: 2, US: 1	India: 2 + 1 = 3
B	India: 1, US: 3	US: 1 + 3 = 4

Local COUNT results become a global SUM over the same group key

Averages Need Sums and Counts

Worker	Values	Partial state
A	10, 10	sum = 20, count = 2
B	90	sum = 90, count = 1

```
AVG = (20 + 90) / (2 + 1) = 36.67
(10 + 90) / 2 = 50    -- wrong: unequal group sizes
```

An aggregate is distributed by preserving the state needed for a correct merge

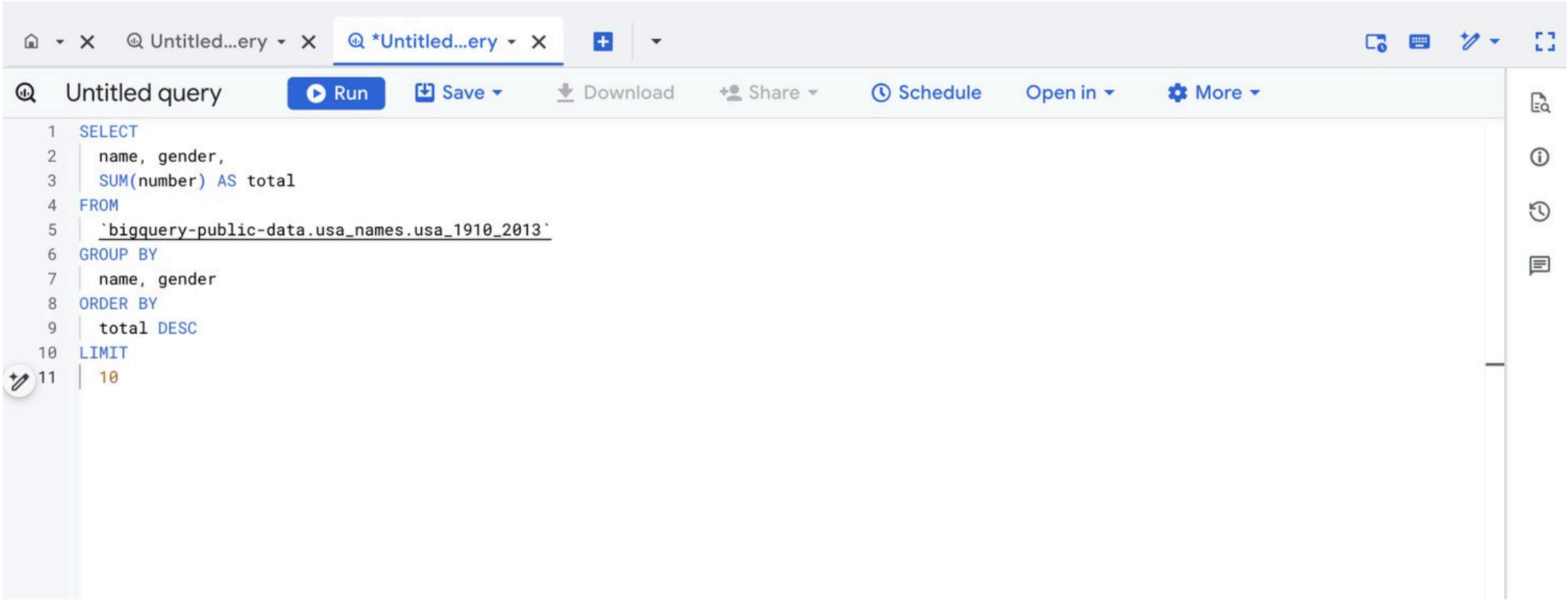
Distinct Counts Need More State

COUNT can merge scalars; exact DISTINCT must account for overlap

Worker	Product IDs	Local distinct count
A	P1, P2	2
B	P1, P3	2
Merge	P1, P2, P3	3, rather than 2 + 2

A correct distributed aggregate must preserve enough information to merge overlapping inputs

SQL over Distributed Execution



The user writes SQL; the service plans and runs distributed analytical work

Facts and Dimensions

fact_sales: one row per sale

sale_id	sale_date	product_id	amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

dim_product: one row per product

product_id	name
P1	Chocolate
P2	Ice-cream

product_id links each sale to one product description.

Two sales of P1 share one product description; each fact retains its sale date

Grain of an Analytical Table

Detail: one row per sale

Sale	Date	Product	Amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

One row per product

Product	Total
P1	700
P2	300

One row per date

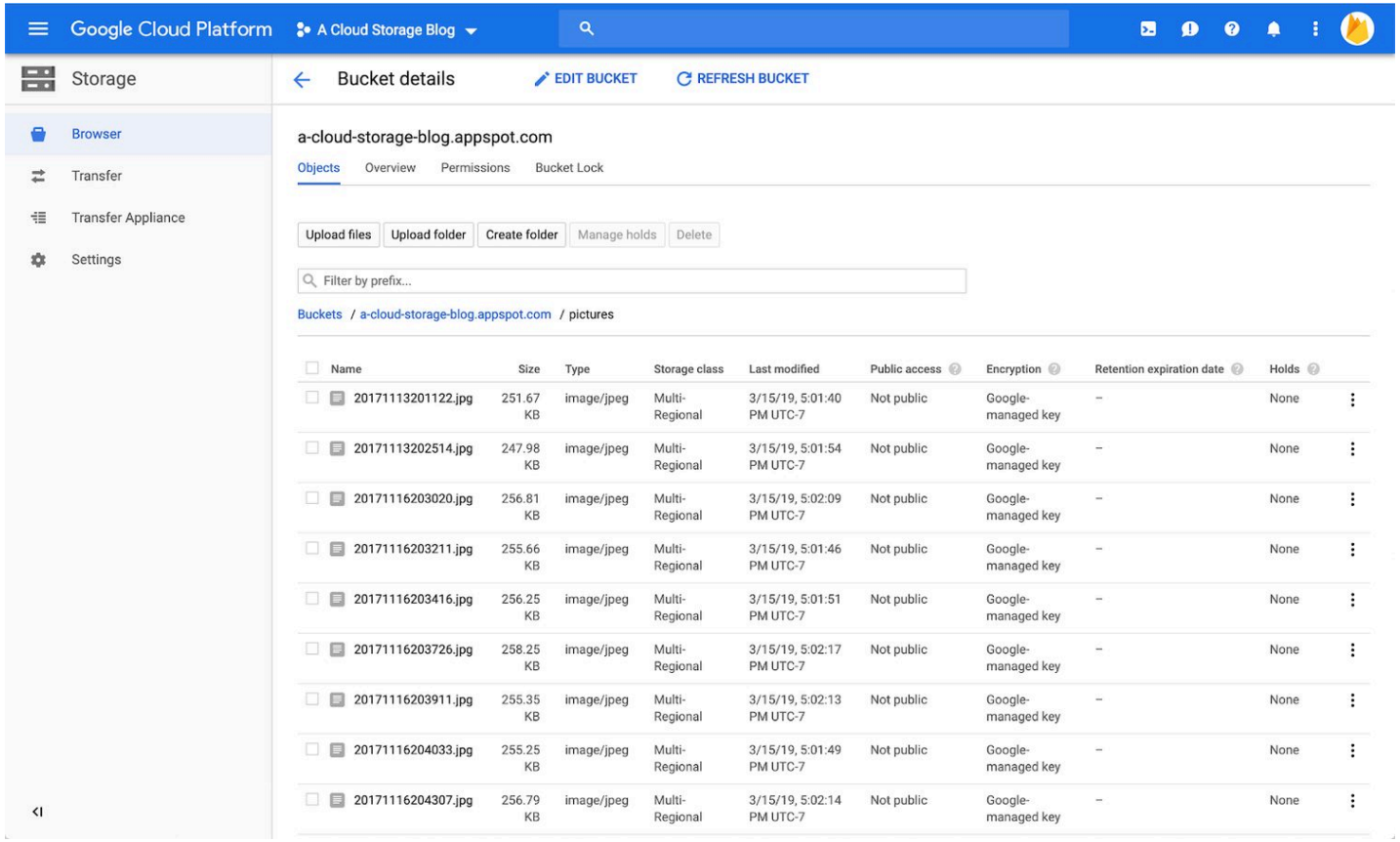
Date	Total
Oct 2	500
Oct 3	500

Both summaries use four dated sales; grouping changes what one result row represents

03 · Lakes and Lakehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

Why Keep a Data Lake



Source files can remain in object storage before analytical tables are defined

Warehouse, Mart and Lake

Store	Organizing unit	Typical purpose
Warehouse	Integrated analytical tables	Consistent organizational analysis
Data mart	A subject or department subset	Focused reports
Lake	Files and datasets in varied formats	Preserve and reuse diverse inputs

A mart narrows the analytical scope; a lake broadens the retained source material

Schema on Read

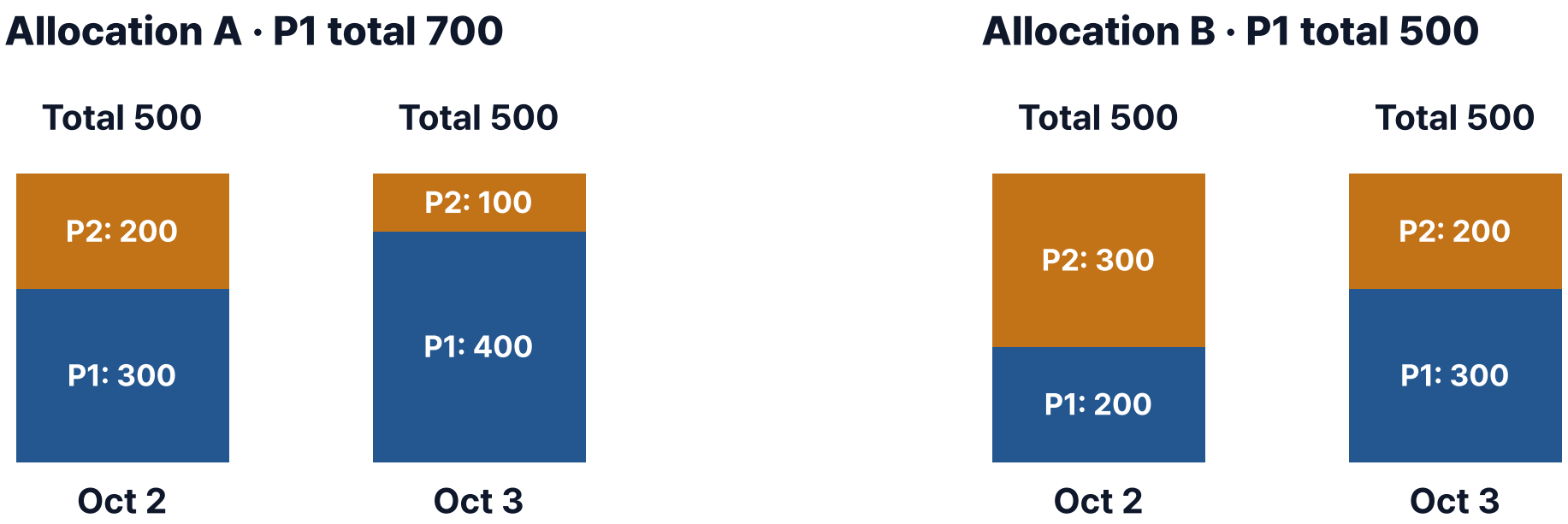
Schema on read applies structure at use time; schema on write validates it when loading



Files can arrive first; structure and types are still required when the data is used

Keep the Grain You May Need

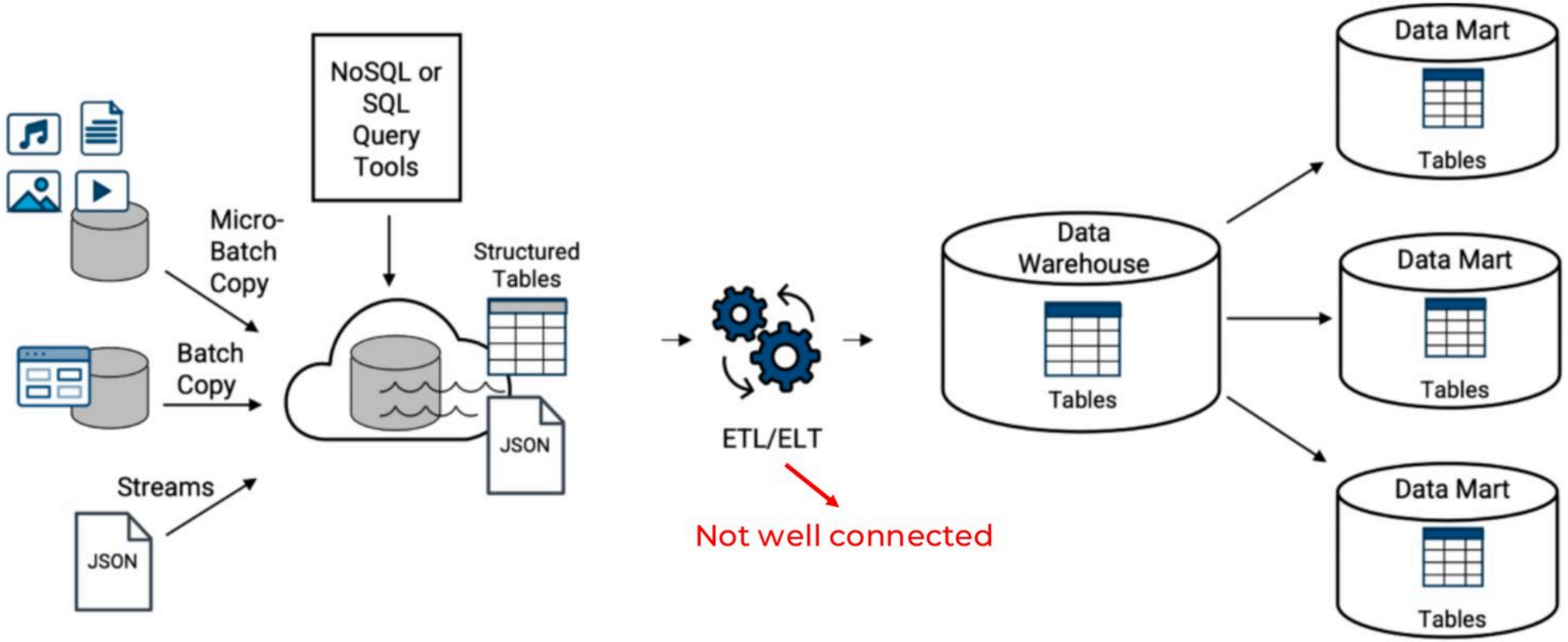
Grain determines which future questions remain answerable



Daily totals: 500 / 500 in BOTH cases; the product allocation is not recoverable

Two different product allocations produce identical daily totals

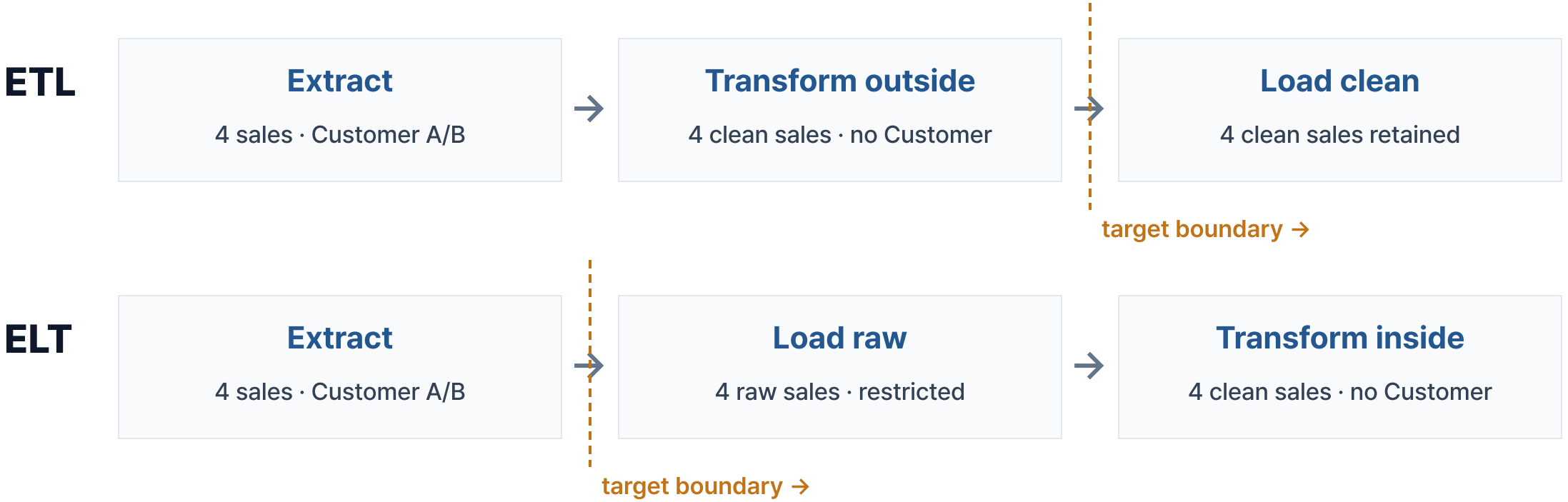
From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)



Same source archive, four clean sale rows; only the transformation location changes

Choosing ETL or ELT

Condition	Typical choice	Reason
Sensitive fields must stay outside the target	ETL	Remove them before crossing the boundary
Target compute is limited	ETL	Transform using upstream processing
Restricted raw area and scalable SQL compute	ELT	Load detail, then build analytical models inside

Choose the location by data boundaries and compute; decide detail retention separately

BigQuery as an ELT Platform

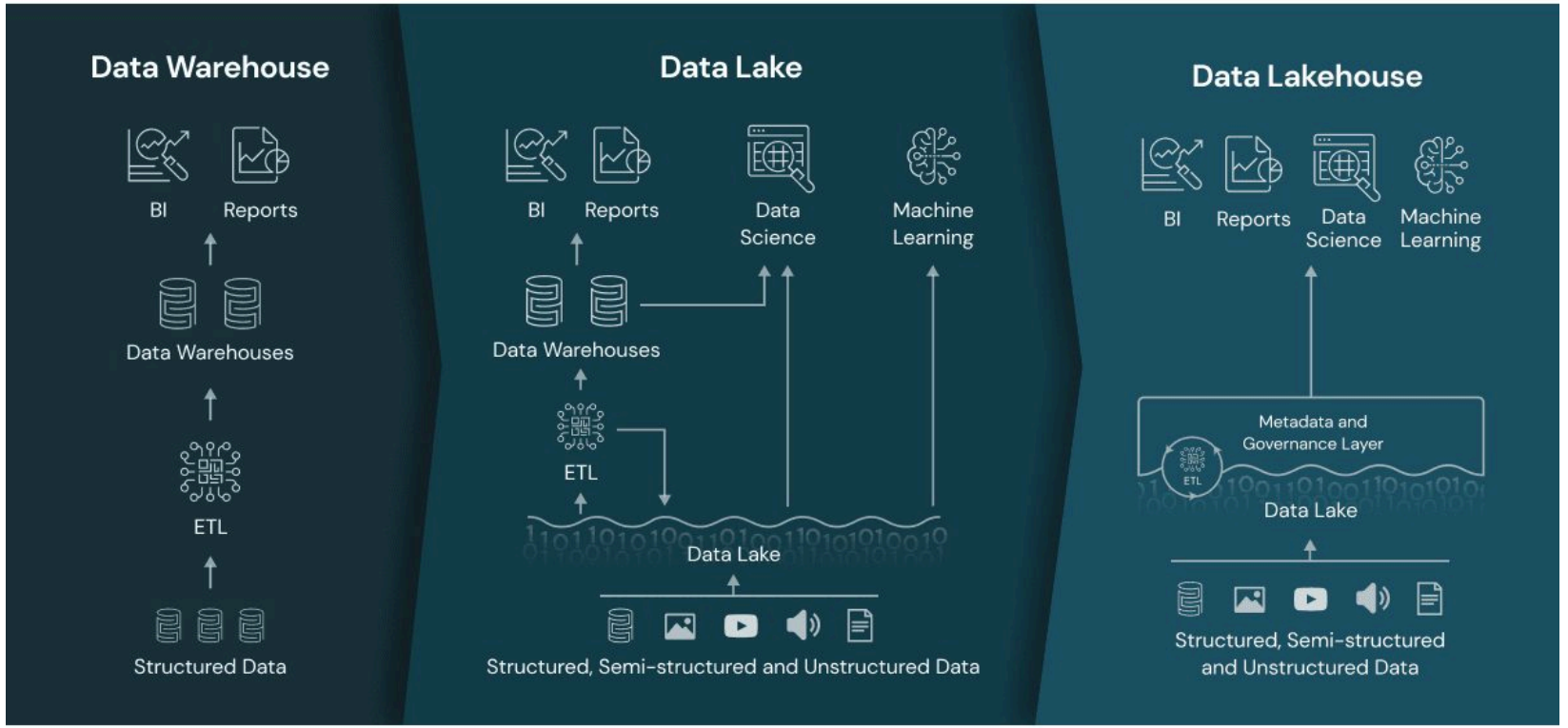
Implementation pattern using BigQuery capabilities; not a recorded production run

Step	Concrete BigQuery object	Evidence to inspect
Load	Restricted raw.sales, partitioned by sale_date	Loaded date and source version
Transform	SQL over raw.sales → staging.daily_sales	One row per date; counts and totals reconcile
Serve	Validated mart.daily_sales table or view	Published date; access granted to report readers

The same sale_date follows load → transform → checks → publication → retry.

The analytical table, checks and retry all refer to the same source date

Why a Lakehouse



Lakehouse adds table management over lake files for shared analytical use

What Makes a Lakehouse Work

Layer	Mechanism	Reason
Table management	Transactions and versioned metadata	Readers see a consistent table
Query execution	Columnar files, pruning and statistics	SQL scans avoid unrelated data
Shared access	SQL and DS/ML use the same snapshot	Fewer independent data copies

Table transactions, query execution and shared data access turn files into managed tables

Case study: Iceberg Table Commits



Write complete files and metadata, then atomically replace the current table metadata reference

Iceberg commits a table version rather than discovering all files in a changing directory

Concurrent Changes Need Validation

Compaction assumption	Concurrent change	Required action
file_a and file_b still belong to the current table	Another writer removes or rewrites file_a	Reject this stale compaction
An append can be applied to the current table	Another writer commits unrelated new files	Rebase metadata and retry when valid

A safe retry checks whether the assumptions behind the prepared table change still hold

Consistency and Analytical Correctness

Guarantee	It protects	It does not establish
Object atomicity	A complete object is visible	Which objects form the table
Table snapshot	One committed table version	Correct joins, units or business definitions
Data validation	Explicit properties of produced data	Every possible property of the analysis

A consistent snapshot can contain incorrect data; analytical meaning still requires validation

Tools Around a Lakehouse

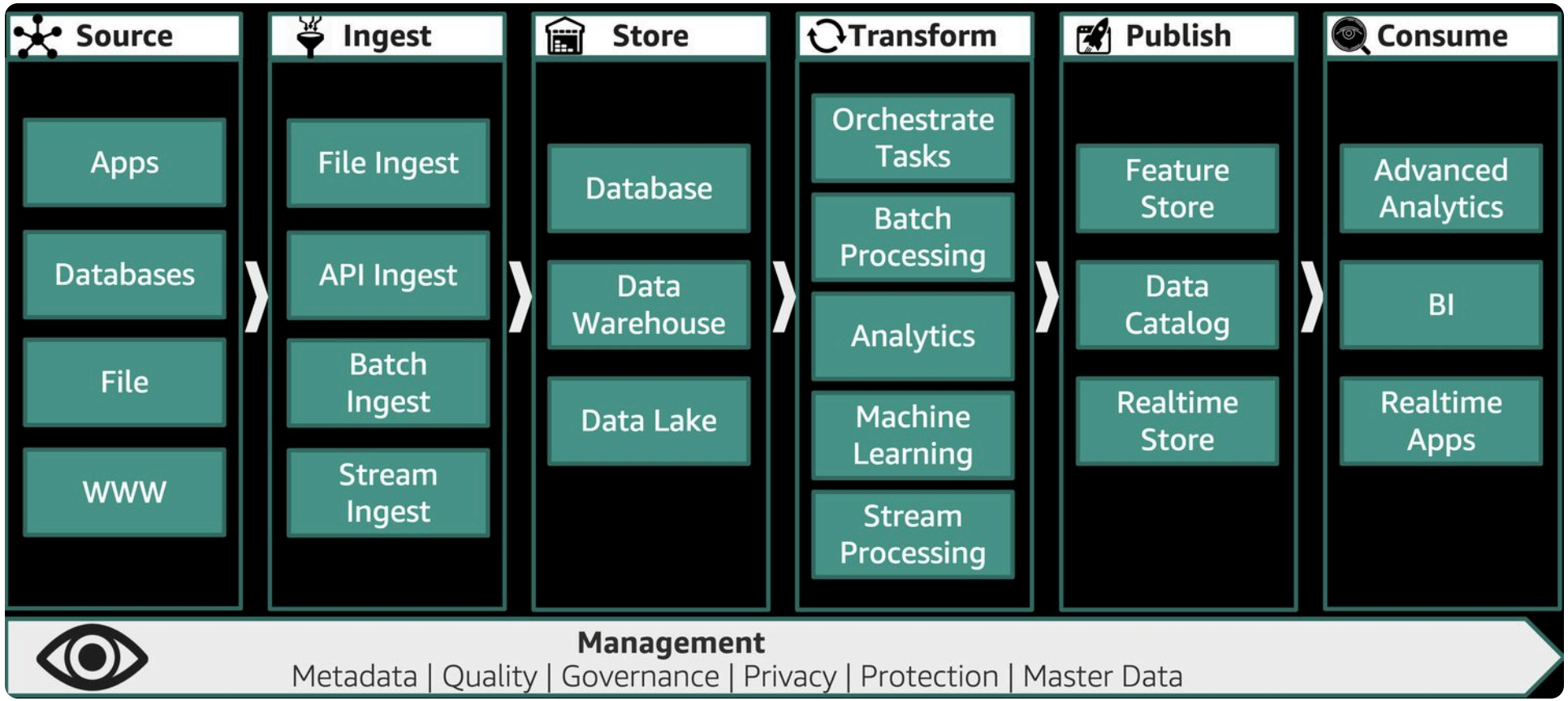
Compute and platform	**Spark, Databricks**	run transformations and queries
Table format	**Iceberg, Delta Lake**	manage table metadata and changes
Columnar file format	**Parquet**	encode values in files
Object storage	**S3, Cloud Storage**	store the data files

Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Data Orchestration



A successful query can still run before its required data is ready

Airflow at Airbnb

2014

started at Airbnb by M. Beauchemin

2015

announced as open source

2016

Apache incubation

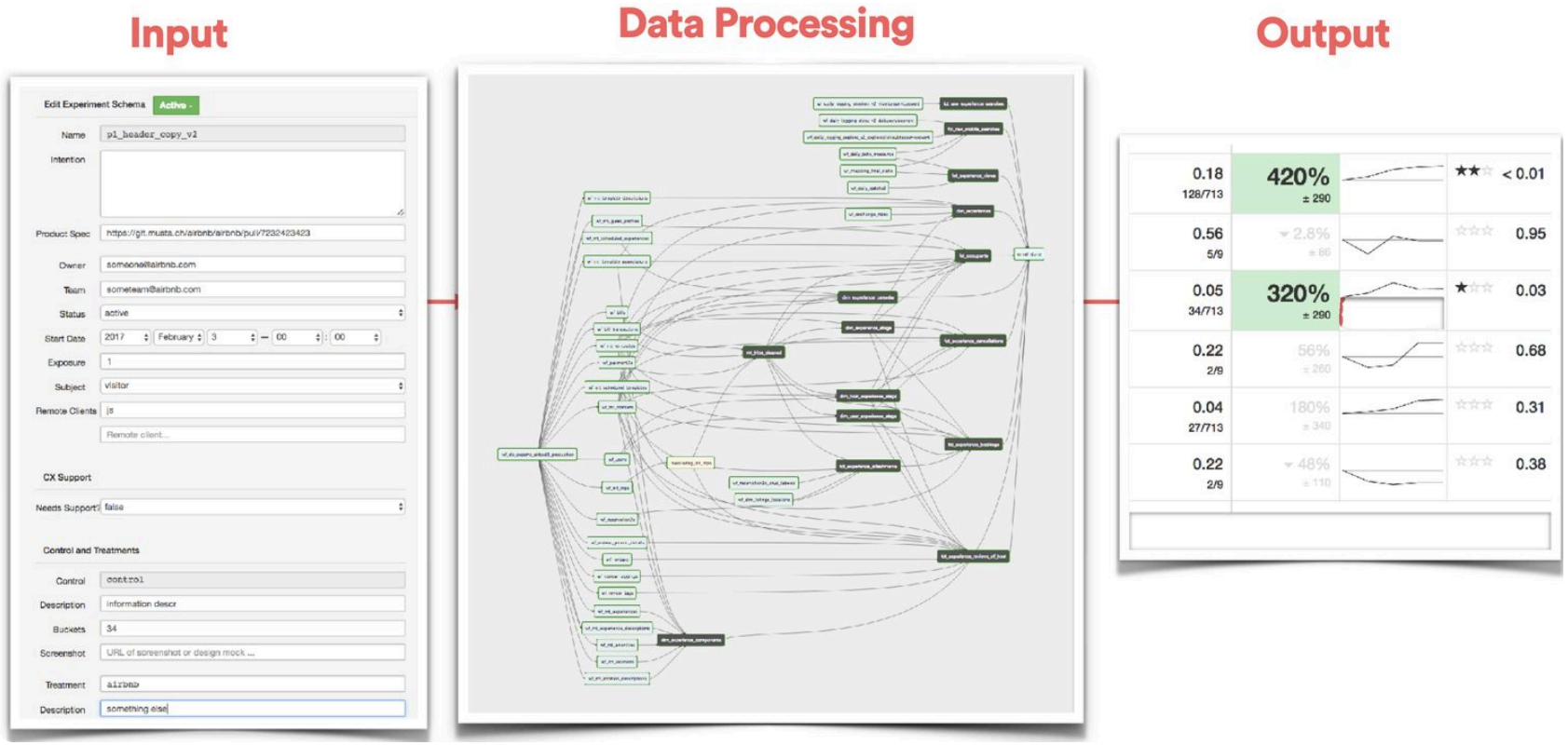
2019

Apache top-level project

Airflow grew from a company workflow tool into an Apache project

Case study: Airbnb Experiment Analytics

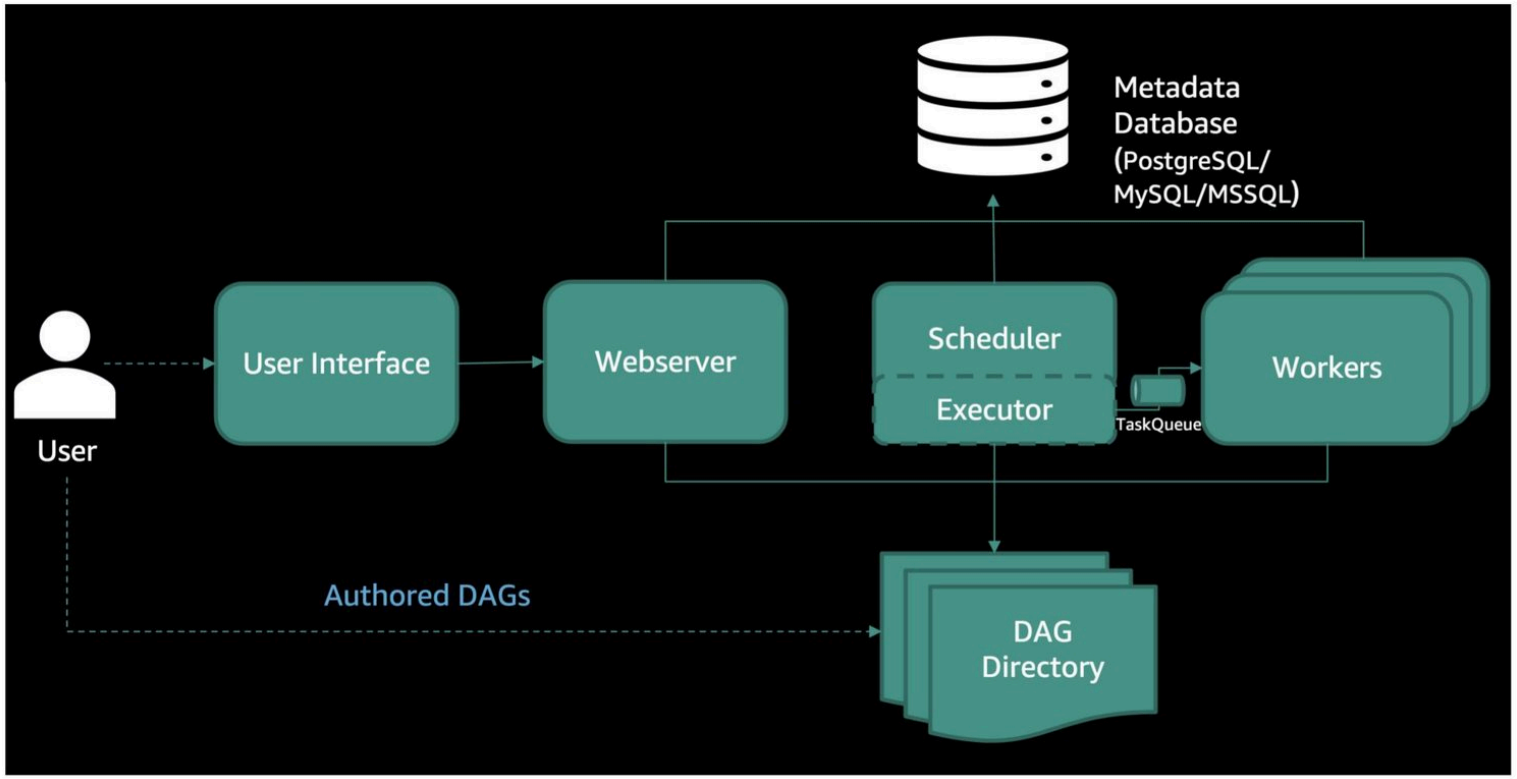
Historical Airbnb workflow: configuration, computation and experiment metrics



Experiment configuration drives a dependency graph that produces analytical results

Airflow Control and Workers

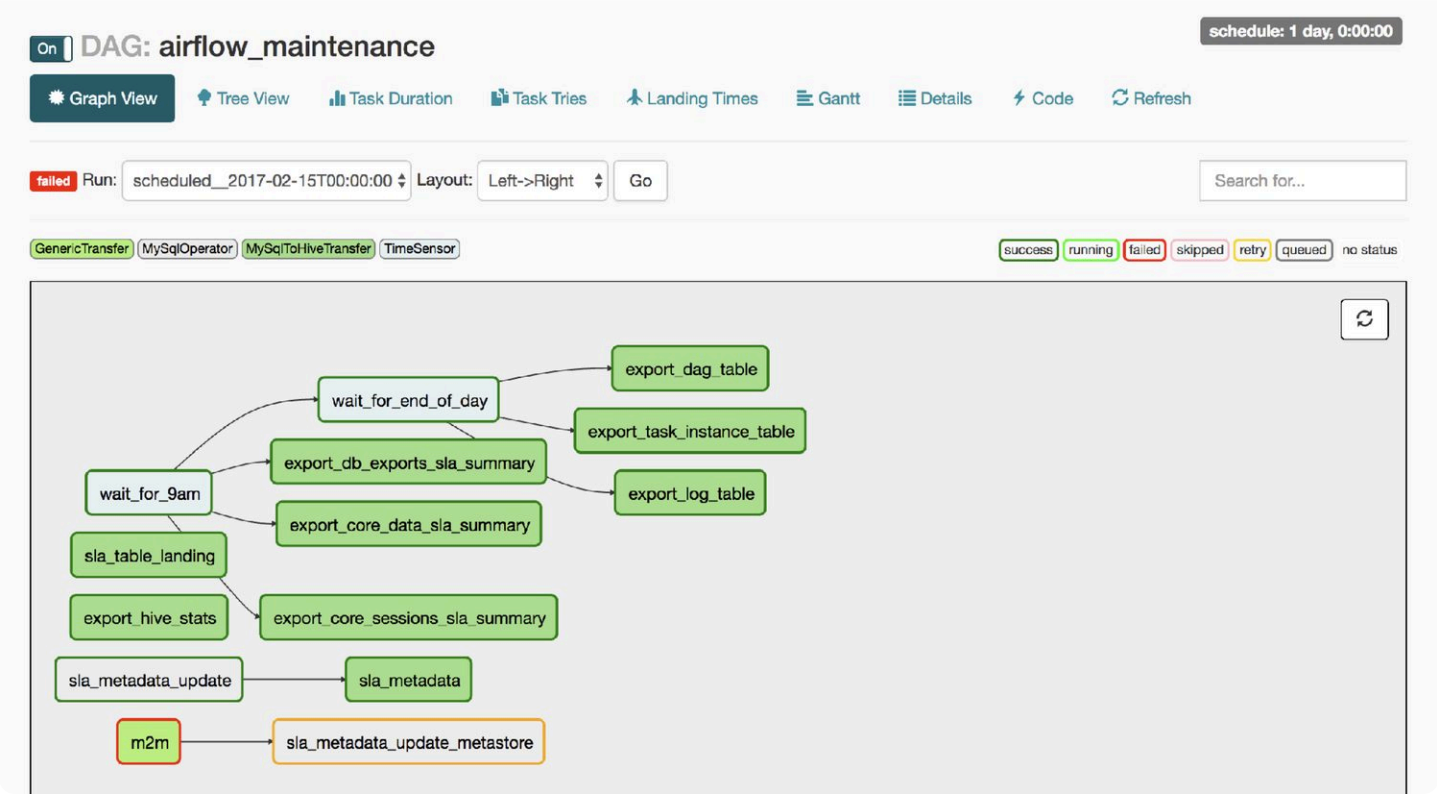
Historical architecture: focus on the responsibilities, rather than deployment details



Scheduling, task execution and recorded state are distinct responsibilities

Tasks and Dependencies

A Directed Acyclic Graph (DAG) defines tasks and their dependencies



- **wait_for_end_of_day**: input condition
- **export_dag_table** and **export_log_table**: parallel branches
- Edges constrain readiness; they do not carry data rows

Independent branches can run in parallel; dependent tasks wait for their upstream work

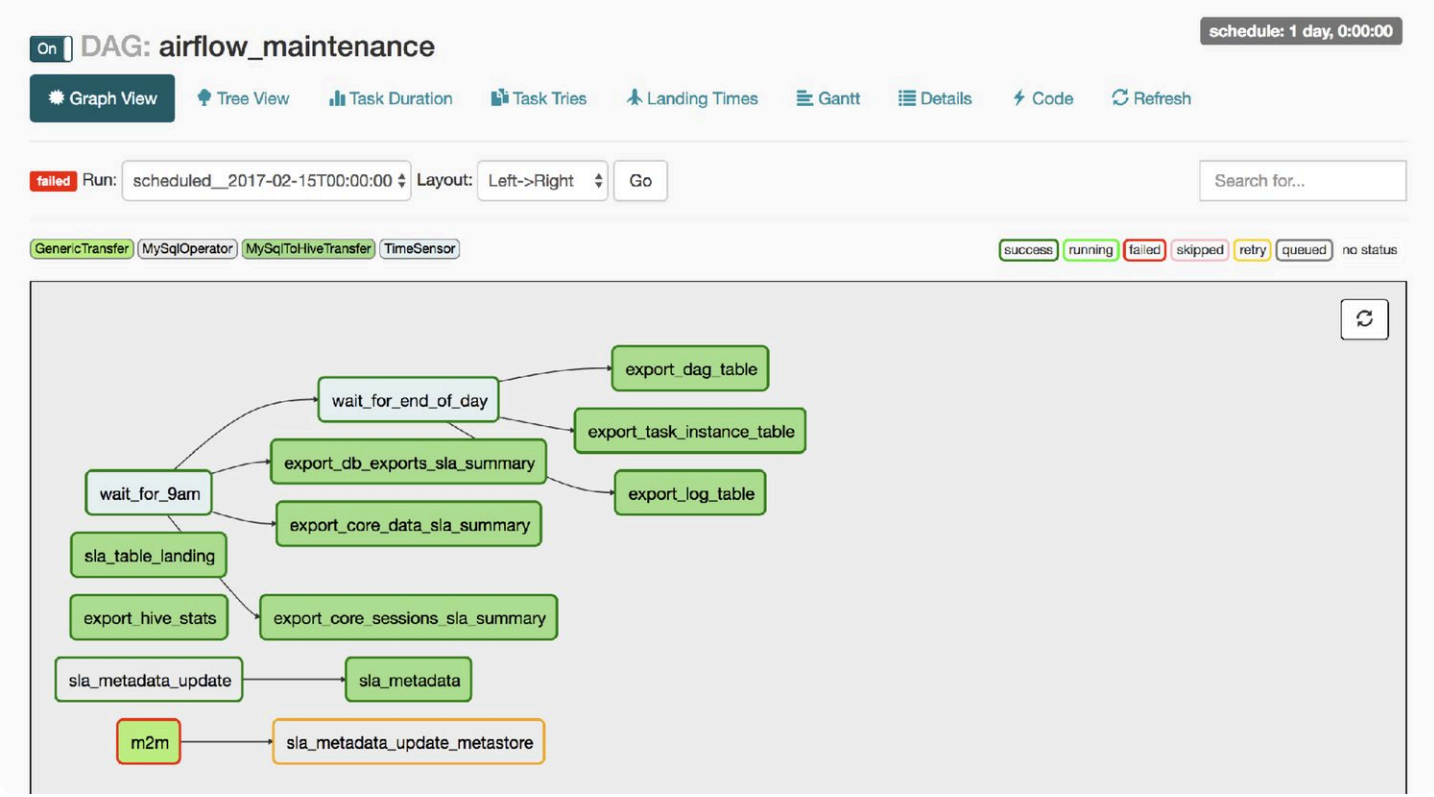
Input Readiness and Task Types

Concept	Role	Example
Task	A concrete unit of work in the DAG	Load the sales partition
Operator	A reusable definition for a task	Submit a warehouse SQL query
Sensor	A task that waits for a condition	Wait for the producer to publish a complete input

A sensor waits for an input contract; a clock time alone does not prove readiness

An Airflow Run

Same maintenance DAG and run as the graph; follow the named task states

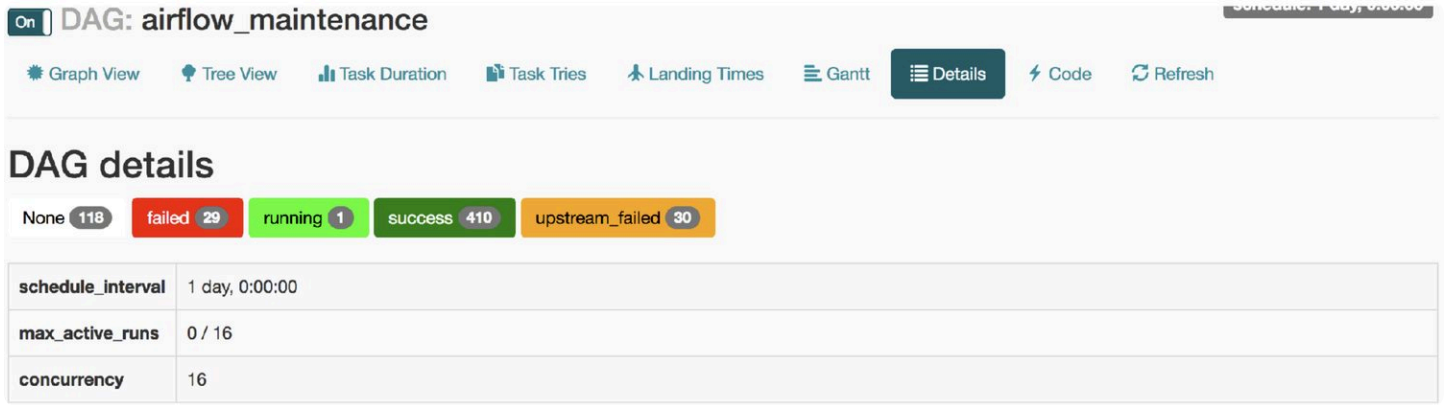


- **Run:** scheduled, 2017-02-15
- **m2m:** yellow, waiting to retry
- **sla_metadata_update_metastore:** orange upstream_failed
- Green exports succeeded; this run still failed

Succeeded exports coexist with retry and upstream_failed states in one failed run

Case study: Airflow Workflow Health

Same maintenance DAG; these totals span task instances, not just the preceding run



- **failed 29**: task failures
- **upstream_failed 30**: blocked by predecessors
- **running 1 / success 410**: other recorded states
- **concurrency 16**: capacity limit

Dependency readiness, execution capacity and recorded task states answer different operational questions

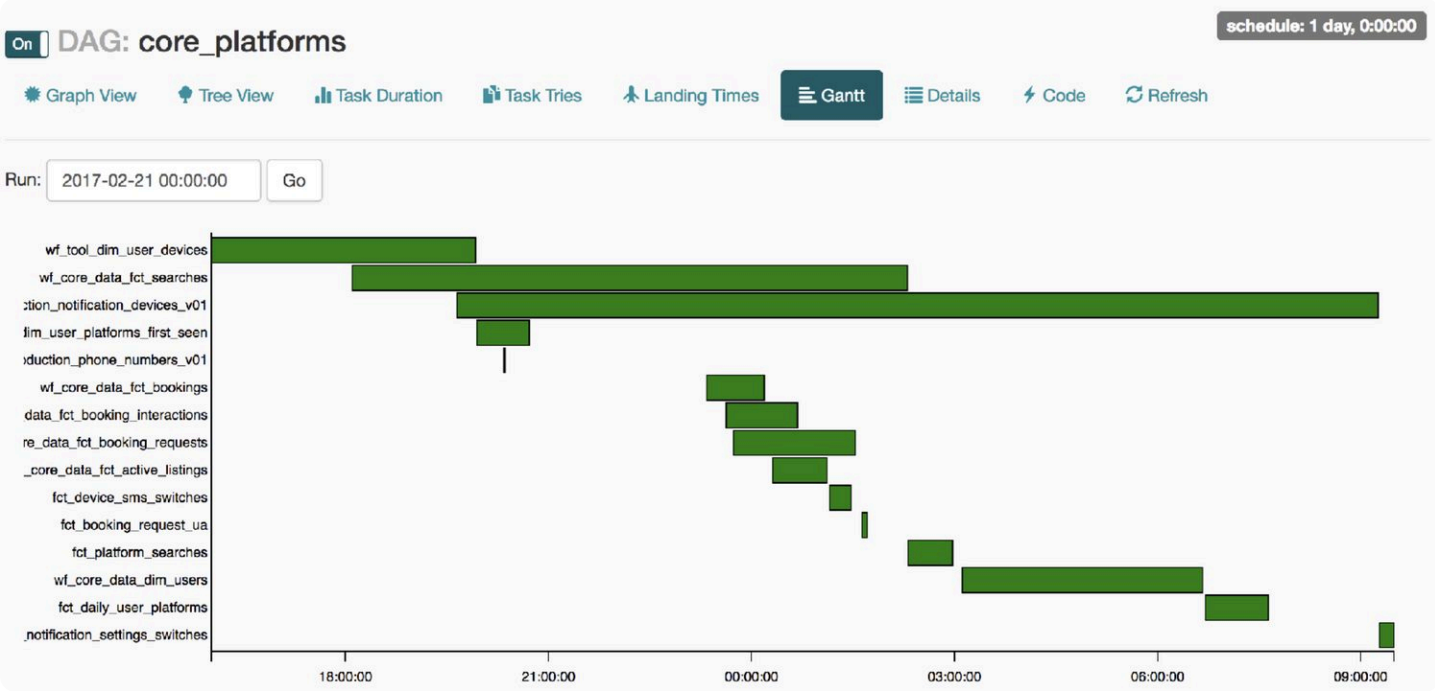
Data Checks Before Publication

Contract	Example check	Failure caught
Stable result grain	One row per intended date or entity	Duplicate report rows
Valid joins	Expected key uniqueness and match coverage	Fan-out or missing dimension matches
Reconcile to source	Compare counts and totals over the same interval	Dropped or repeated inputs
Publish after validation	Make checks upstream of publication	A successful task publishing invalid data

Task success records execution; checks must explicitly test the intended analytical contracts

Read Execution Time in Gantt

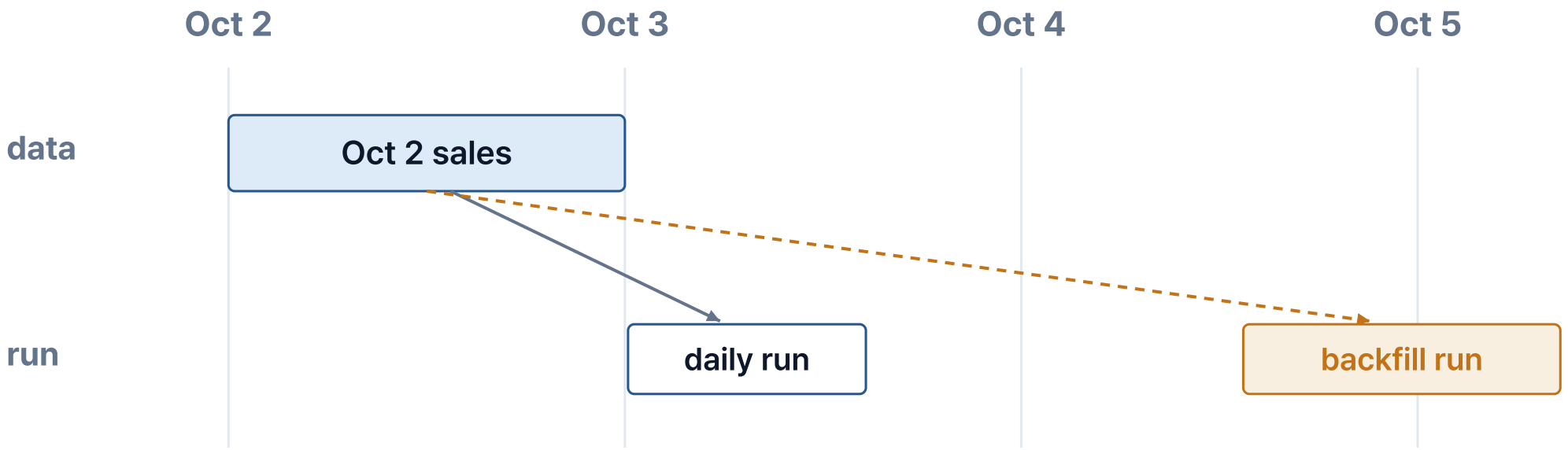
Separate case: core_platforms timing, not the maintenance run shown earlier



- **core_platforms**: a different workflow
- Overlapping bars show concurrent execution
- Long bars identify tasks to investigate
- Use this workflow's DAG to establish its critical path

Dependencies bound completion; resource limits can delay ready tasks further

Data Interval and Run Time



The date being recomputed can differ from the time when the task runs

Retry and Backfill

Mechanism	Purpose	Required design
Retry	Run a failed or uncertain task again	Repeat without duplicating side effects
Backfill	Compute a selected historical interval	Use that interval, rather than the current time
Safe publication	Replace the prepared output after validation	Keep input version, target interval and commit boundary explicit

Orchestration schedules re-execution; the task must define safe, repeatable data changes

Replace a Fixed Data Interval

Idempotence means repeating the same operation preserves the intended result

Step	BigQuery pattern for Oct 2	Verification
Read	Fixed source version in raw.sales for Oct 2	Source date and version match the intended backfill
Prepare	Build staging.daily_sales for Oct 2	Validate grain, keys, counts and totals
Publish	In one transaction: replace Oct 2 in mart.daily_sales	Commit succeeded; other dates preserved

Retry the same replacement; reconcile Oct 2 after commit instead of appending again.

Fix the input and target interval, then replace its prepared result without accumulating duplicates

Failure Around the Commit Boundary

Failure point	Visible result	What a retry must establish
Before publication	Previous complete version remains current	Prepare and validate again
After commit, before acknowledgment	New version may already be current	Check committed state before repeating side effects
Source changed before retry	Recomputed result may legitimately differ	Choose the intended source version explicitly

A failed response can hide a successful commit; retry safety depends on the publication design

Diagnosing an Analytical Pipeline

Observed symptom	Question to ask	Relevant mechanism
Correct result but excessive scans	Which columns and partitions were read?	Projection, pruning and columnar encoding
Complete table with incorrect totals	Did joins and result grain preserve the intended meaning?	Keys, grain and data checks
A retry changes or duplicates prior output	Which input version and target interval were used?	Fixed-scope replacement and commit state

Locate the failed guarantee before changing products or allocating more execution resources

Takeaway: Tools Have Different Responsibilities

1. **Managed analytics** hides infrastructure while retaining storage and execution choices
2. **OLTP / OLAP** usually favor row / column stores; lakes retain inputs and lakehouses manage tables
3. **Airflow** coordinates task dependencies, input readiness and repeatable execution